A number of useful shortcuts can be applied when using asymptotic notation. First:

$$O(n^{c_1}) \subset O(n^{c_2}) \ ,$$

for any $c_1 < c_2$. Second: For any constants $a, b, c > 0$,

$$O(a) \subset O(\log n) \subset O(n^b) \subset O(c^n) \ .$$

These inclusion relations can be multiplied by any positive value, and they still hold. For example, multiplying by n yields:

$$O(n) \subset O(n \log n) \subset O(n^{1+b}) \subset O(nc^n) \ .$$

Continuing in a long and distinguished tradition, we will abuse this notation by writing things like $f_1(n) = O(f(n))$ when what we really mean is $f_1(n) \in O(f(n))$. We will also make statements like "the running time of this operation is $O(f(n))$" when this statement should be "the running time of this operation is *a member of* $O(f(n))$." These shortcuts are mainly to avoid awkward language and to make it easier to use asymptotic notation within strings of equations.

A particularly strange example of this occurs when we write statements like

$$T(n) = 2 \log n + O(1) \ .$$

Again, this would be more correctly written as

$$T(n) \leq 2 \log n + [\text{some member of } O(1)] \ .$$

The expression $O(1)$ also brings up another issue. Since there is no variable in this expression, it may not be clear which variable is getting arbitrarily large. Without context, there is no way to tell. In the example above, since the only variable in the rest of the equation is n, we can assume that this should be read as $T(n) = 2 \log n + O(f(n))$, where $f(n) = 1$.

Big-Oh notation is not new or unique to computer science. It was used by the number theorist Paul Bachmann as early as 1894, and is immensely useful for describing the running times of computer algorithms. Consider the following piece of code: One execution of this method involves

- 1 assignment ($int\, i \leftarrow 0$),

- $n+1$ comparisons ($i < n$),
- n increments ($i++$),
- n array offset calculations ($a[i]$), and
- n indirect assignments ($a[i] \leftarrow i$).

So we could write this running time as

$$T(n) = a + b(n+1) + cn + dn + en \ ,$$

where a, b, c, d, and e are constants that depend on the machine running the code and represent the time to perform assignments, comparisons, increment operations, array offset calculations, and indirect assignments, respectively. However, if this expression represents the running time of two lines of code, then clearly this kind of analysis will not be tractable to complicated code or algorithms. Using big-Oh notation, the running time can be simplified to

$$T(n) = O(n) \ .$$

Not only is this more compact, but it also gives nearly as much information. The fact that the running time depends on the constants a, b, c, d, and e in the above example means that, in general, it will not be possible to compare two running times to know which is faster without knowing the values of these constants. Even if we make the effort to determine these constants (say, through timing tests), then our conclusion will only be valid for the machine we run our tests on.

Big-Oh notation allows us to reason at a much higher level, making it possible to analyze more complicated functions. If two algorithms have the same big-Oh running time, then we won't know which is faster, and there may not be a clear winner. One may be faster on one machine, and the other may be faster on a different machine. However, if the two algorithms have demonstrably different big-Oh running times, then we can be certain that the one with the smaller running time will be faster *for large enough values of n*.

An example of how big-Oh notation allows us to compare two different functions is shown in Figure 1.5, which compares the rate of growth

of $f_1(n) = 15n$ versus $f_2(n) = 2n \log n$. It might be that $f_1(n)$ is the running time of a complicated linear time algorithm while $f_2(n)$ is the running time of a considerably simpler algorithm based on the divide-and-conquer paradigm. This illustrates that, although $f_1(n)$ is greater than $f_2(n)$ for small values of n, the opposite is true for large values of n. Eventually $f_1(n)$ wins out, by an increasingly wide margin. Analysis using big-Oh notation told us that this would happen, since $O(n) \subset O(n \log n)$.

In a few cases, we will use asymptotic notation on functions with more than one variable. There seems to be no standard for this, but for our purposes, the following definition is sufficient:

$$O(f(n_1,\ldots,n_k)) = \left\{ \begin{array}{l} g(n_1,\ldots,n_k) : \text{there exists } c > 0, \text{ and } z \text{ such that} \\ \quad g(n_1,\ldots,n_k) \leq c \cdot f(n_1,\ldots,n_k) \\ \quad\quad \text{for all } n_1,\ldots,n_k \text{ such that } g(n_1,\ldots,n_k) \geq z \end{array} \right\} .$$

This definition captures the situation we really care about: when the arguments $n_1,\ldots,n_k$ make g take on large values. This definition also agrees with the univariate definition of $O(f(n))$ when $f(n)$ is an increasing function of n. The reader should be warned that, although this works for our purposes, other texts may treat multivariate functions and asymptotic notation differently.

1.3.4 Randomization and Probability

Some of the data structures presented in this book are *randomized*; they make random choices that are independent of the data being stored in them or the operations being performed on them. For this reason, performing the same set of operations more than once using these structures could result in different running times. When analyzing these data structures we are interested in their average or *expected* running times.

Formally, the running time of an operation on a randomized data structure is a random variable, and we want to study its *expected value*. For a discrete random variable X taking on values in some countable universe U, the expected value of X, denoted by $\mathrm{E}[X]$, is given by the formula

$$\mathrm{E}[X] = \sum_{x \in U} x \cdot \Pr\{X = x\} .$$

Here $\Pr\{\mathcal{E}\}$ denotes the probability that the event $\mathcal{E}$ occurs. In all of the

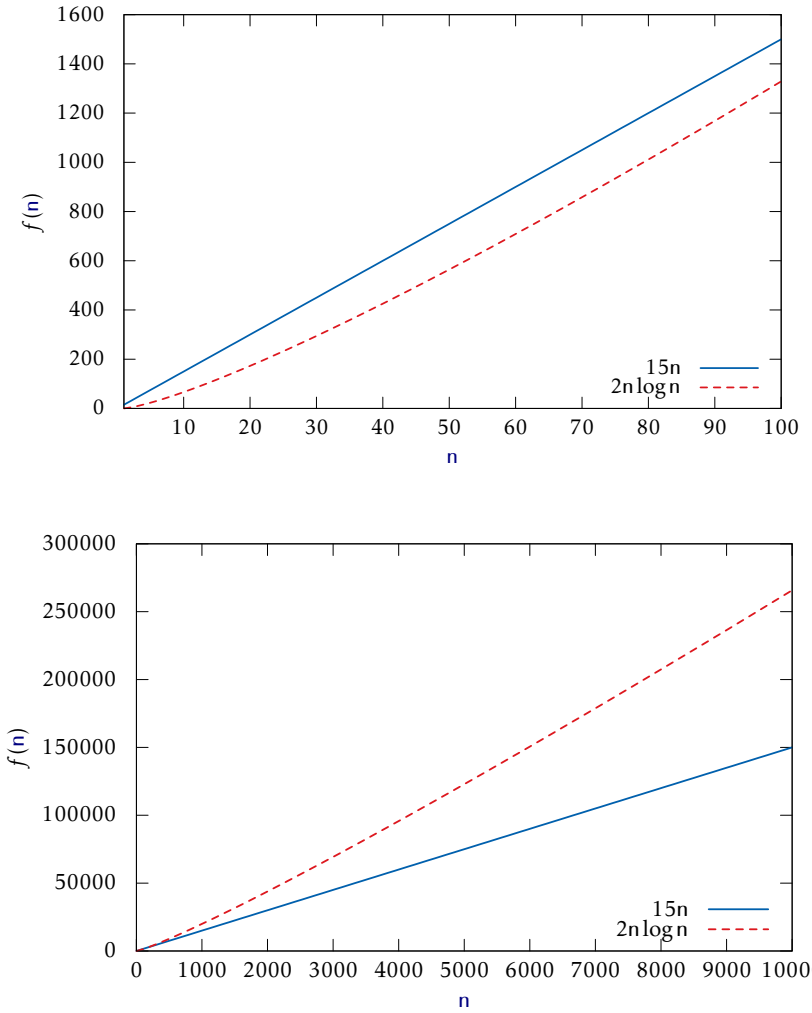

Figure 1.5: Plots of $15n$ versus $2n \log n$.

examples in this book, these probabilities are only with respect to the random choices made by the randomized data structure; there is no assumption that the data stored in the structure, nor the sequence of operations performed on the data structure, is random.

One of the most important properties of expected values is *linearity of expectation*. For any two random variables X and Y,

$$\mathrm{E}[X+Y] = \mathrm{E}[X] + \mathrm{E}[Y] \ .$$

More generally, for any random variables $X_1, \ldots, X_k$,

$$\mathrm{E}\left[\sum_{i=1}^{k} X_k\right] = \sum_{i=1}^{k} \mathrm{E}[X_i] \ .$$

Linearity of expectation allows us to break down complicated random variables (like the left hand sides of the above equations) into sums of simpler random variables (the right hand sides).

A useful trick, that we will use repeatedly, is defining *indicator random variables*. These binary variables are useful when we want to count something and are best illustrated by an example. Suppose we toss a fair coin k times and we want to know the expected number of times the coin turns up as heads. Intuitively, we know the answer is $k/2$, but if we try to prove it using the definition of expected value, we get

$$\begin{aligned}
\mathrm{E}[X] &= \sum_{i=0}^{k} i \cdot \Pr\{X = i\} \\
&= \sum_{i=0}^{k} i \cdot \binom{k}{i} / 2^k \\
&= k \cdot \sum_{i=0}^{k-1} \binom{k-1}{i} / 2^k \\
&= k/2 \ .
\end{aligned}$$

This requires that we know enough to calculate that $\Pr\{X = i\} = \binom{k}{i}/2^k$, and that we know the binomial identities $i\binom{k}{i} = k\binom{k-1}{i}$ and $\sum_{i=0}^{k}\binom{k}{i} = 2^k$.

Using indicator variables and linearity of expectation makes things

much easier. For each $i \in \{1,\ldots,k\}$, define the indicator random variable

$$I_i = \begin{cases} 1 & \text{if the } i\text{th coin toss is heads} \\ 0 & \text{otherwise.} \end{cases}$$

Then

$$\mathrm{E}[I_i] = (1/2)1 + (1/2)0 = 1/2 \ .$$

Now, $X = \sum_{i=1}^{k} I_i$, so

$$\begin{aligned} \mathrm{E}[X] &= \mathrm{E}\left[\sum_{i=1}^{k} I_i\right] \\ &= \sum_{i=1}^{k} \mathrm{E}[I_i] \\ &= \sum_{i=1}^{k} 1/2 \\ &= k/2 \ . \end{aligned}$$

This is a bit more long-winded, but doesn't require that we know any magical identities or compute any non-trivial probabilities. Even better, it agrees with the intuition that we expect half the coins to turn up as heads precisely because each individual coin turns up as heads with a probability of 1/2.

1.4 The Model of Computation

In this book, we will analyze the theoretical running times of operations on the data structures we study. To do this precisely, we need a mathematical model of computation. For this, we use the *w-bit word-RAM* model. RAM stands for Random Access Machine. In this model, we have access to a random access memory consisting of *cells*, each of which stores a w-bit *word*. This implies that a memory cell can represent, for example, any integer in the set $\{0,\ldots,2^w-1\}$.

In the word-RAM model, basic operations on words take constant time. This includes arithmetic operations ($+$, $-$, $\cdot$, $/$, mod), comparisons

($<$, $>$, $=$, $\leq$, $\geq$), and bitwise boolean operations (bitwise-AND, OR, and exclusive-OR).

Any cell can be read or written in constant time. A computer's memory is managed by a memory management system from which we can allocate or deallocate a block of memory of any size we would like. Allocating a block of memory of size k takes $O(k)$ time and returns a reference (a pointer) to the newly-allocated memory block. This reference is small enough to be represented by a single word.

The word-size w is a very important parameter of this model. The only assumption we will make about w is the lower-bound $w \geq \log n$, where n is the number of elements stored in any of our data structures. This is a fairly modest assumption, since otherwise a word is not even big enough to count the number of elements stored in the data structure.

Space is measured in words, so that when we talk about the amount of space used by a data structure, we are referring to the number of words of memory used by the structure. All of our data structures store values of a generic type T, and we assume an element of type T occupies one word of memory.

The data structures presented in this book don't use any special tricks that are not implementable

1.5 Correctness, Time Complexity, and Space Complexity

When studying the performance of a data structure, there are three things that matter most:

Correctness: The data structure should correctly implement its interface.

Time complexity: The running times of operations on the data structure should be as small as possible.

Space complexity: The data structure should use as little memory as possible.

In this introductory text, we will take correctness as a given; we won't consider data structures that give incorrect answers to queries or don't

perform updates properly. We will, however, see data structures that make an extra effort to keep space usage to a minimum. This won't usually affect the (asymptotic) running times of operations, but can make the data structures a little slower in practice.

When studying running times in the context of data structures we tend to come across three different kinds of running time guarantees:

Worst-case running times: These are the strongest kind of running time guarantees. If a data structure operation has a worst-case running time of $f(n)$, then one of these operations *never* takes longer than $f(n)$ time.

Amortized running times: If we say that the amortized running time of an operation in a data structure is $f(n)$, then this means that the cost of a typical operation is at most $f(n)$. More precisely, if a data structure has an amortized running time of $f(n)$, then a sequence of m operations takes at most $mf(n)$ time. Some individual operations may take more than $f(n)$ time but the average, over the entire sequence of operations, is at most $f(n)$.

Expected running times: If we say that the expected running time of an operation on a data structure is $f(n)$, this means that the actual running time is a random variable (see Section 1.3.4) and the expected value of this random variable is at most $f(n)$. The randomization here is with respect to random choices made by the data structure.

To understand the difference between worst-case, amortized, and expected running times, it helps to consider a financial example. Consider the cost of buying a house:

Worst-case versus amortized cost: Suppose that a home costs \$120 000. In order to buy this home, we might get a 120 month (10 year) mortgage with monthly payments of \$1 200 per month. In this case, the worst-case monthly cost of paying this mortgage is \$1 200 per month.

If we have enough cash on hand, we might choose to buy the house outright, with one payment of \$120 000. In this case, over a period of 10 years, the amortized monthly cost of buying this house is

$$\$120\,000/120 \text{ months} = \$1\,000 \text{ per month} \ .$$

This is much less than the $1 200 per month we would have to pay if we took out a mortgage.

Worst-case versus expected cost: Next, consider the issue of fire insurance on our $120 000 home. By studying hundreds of thousands of cases, insurance companies have determined that the expected amount of fire damage caused to a home like ours is $10 per month. This is a very small number, since most homes never have fires, a few homes may have some small fires that cause a bit of smoke damage, and a tiny number of homes burn right to their foundations. Based on this information, the insurance company charges $15 per month for fire insurance.

Now it's decision time. Should we pay the $15 worst-case monthly cost for fire insurance, or should we gamble and self-insure at an expected cost of $10 per month? Clearly, the $10 per month costs less *in expectation*, but we have to be able to accept the possibility that the *actual cost* may be much higher. In the unlikely event that the entire house burns down, the actual cost will be $120 000.

These financial examples also offer insight into why we sometimes settle for an amortized or expected running time over a worst-case running time. It is often possible to get a lower expected or amortized running time than a worst-case running time. At the very least, it is very often possible to get a much simpler data structure if one is willing to settle for amortized or expected running times.

1.6 Code Samples

The code samples in this book are written in pseudocode. These should be easy enough to read for anyone who has any programming experience in any of the most common programming languages of the last 40 years. To get an idea of what the pseudocode in this book looks like, here is a function that computes the average of an array, a:

```
average(a)
    s ← 0
    for i in 0, 1, 2, ..., length(a) − 1 do
```

```
        s ← s + a[i]
    return s/length(a)
```

As this code illustrates, assigning to a variable is done using the ← notation. We use the convention that the length of an array, a, is denoted by length(a) and array indices start at zero, so that $0, 1, 2, \ldots,$ length(a) $- 1$ are the valid indices for a. To shorten code, and sometimes make it easier to read, our pseudocode allows for (sub)array assignments. The following two functions are equivalent:

```
left_shift_a(a)
    for i in 0, 1, 2, ..., length(a) − 2 do
        a[i] ← a[i + 1]
    a[length(a) − 1] ← nil

left_shift_b(a)
    a[0, 1, ..., length(a) − 2] ← a[1, 2, ..., length(a) − 1]
    a[length(a) − 1] ← nil
```

The following code sets all the values in an array to zero:

```
zero(a)
    a[0, 1, ..., length(a) − 1] ← 0
```

When analyzing the running time of code like this, we have to be careful; statements like $a[0, 1, \ldots,$ length(a) $- 1] \leftarrow 1$ or $a[1, 2, \ldots,$ length(a) $- 1] \leftarrow a[0, 1, \ldots,$ length(a) $- 2]$ do not run in constant time. They run in $O($length(a)$)$ time.

We take similar shortcuts with variable assignments, so that the code $x, y \leftarrow 0, 1$ sets x to zero and y to 1 and the code $x, y \leftarrow y, x$ swaps the values of the variables x and y.

Our pseudocode uses a few operators that may be unfamiliar. As is standard in mathematics, (normal) division is denoted by the / operator. In many cases, we want to do integer division instead, in which case we

use the div operator, so that $a \operatorname{div} b = \lfloor a/b \rfloor$ is the integer part of a/b. So, for example, $3/2 = 1.5$ but $3 \operatorname{div} 2 = 1$. Occasionally, we also use the mod operator to obtain the remainder from integer division, but this will be defined when it is used. Later in the book, we may use some bitwise operators including left-shift ($\ll$), right shift ($\gg$), bitwise and ($\wedge$), and bitwise exclusive-or ($\oplus$).

The pseudocode samples in this book are machine translations of Python code that can be downloaded from the book's website.[2] If you ever encounter an ambiguity in the pseudocode that you can't resolve yourself, then you can always refer to the corresponding Python code. If you don't read Python, the code is also available in Java and C++. If you can't decipher the pseudocode, or read Python, C++, or Java, then you may not be ready for this book.

1.7 List of Data Structures

Tables 1.1 and 1.2 summarize the performance of data structures in this book that implement each of the interfaces, List, USet, and SSet, described in Section 1.2. Figure 1.6 shows the dependencies between various chapters in this book. A dashed arrow indicates only a weak dependency, in which only a small part of the chapter depends on a previous chapter or only the main results of the previous chapter.

1.8 Discussion and Exercises

The List, USet, and SSet interfaces described in Section 1.2 are influenced by the Java Collections Framework [54]. These are essentially simplified versions of the List, Set, Map, SortedSet, and SortedMap interfaces found in the Java Collections Framework.

For a superb (and free) treatment of the mathematics discussed in this chapter, including asymptotic notation, logarithms, factorials, Stirling's approximation, basic probability, and lots more, see the textbook by Leyman, Leighton, and Meyer [50]. For a gentle calculus text that includes

[2] http://opendatastructures.org

List implementations			
	get(i)/set(i,x)	add(i,x)/remove(i)	
ArrayStack	$O(1)$	$O(1+n-i)^{\mathrm{A}}$	§ 2.1
ArrayDeque	$O(1)$	$O(1+\min\{i,n-i\})^{\mathrm{A}}$	§ 2.4
DualArrayDeque	$O(1)$	$O(1+\min\{i,n-i\})^{\mathrm{A}}$	§ 2.5
RootishArrayStack	$O(1)$	$O(1+n-i)^{\mathrm{A}}$	§ 2.6
DLList	$O(1+\min\{i,n-i\})$	$O(1+\min\{i,n-i\})$	§ 3.2
SEList	$O(1+\min\{i,n-i\}/b)$	$O(b+\min\{i,n-i\}/b)^{\mathrm{A}}$	§ 3.3
SkiplistList	$O(\log n)^{\mathrm{E}}$	$O(\log n)^{\mathrm{E}}$	§ 4.3

USet implementations			
	find(x)	add(x)/remove(x)	
ChainedHashTable	$O(1)^{\mathrm{E}}$	$O(1)^{\mathrm{A,E}}$	§ 5.1
LinearHashTable	$O(1)^{\mathrm{E}}$	$O(1)^{\mathrm{A,E}}$	§ 5.2

$^{\mathrm{A}}$ Denotes an *amortized* running time.
$^{\mathrm{E}}$ Denotes an *expected* running time.

Table 1.1: Summary of List and USet implementations.

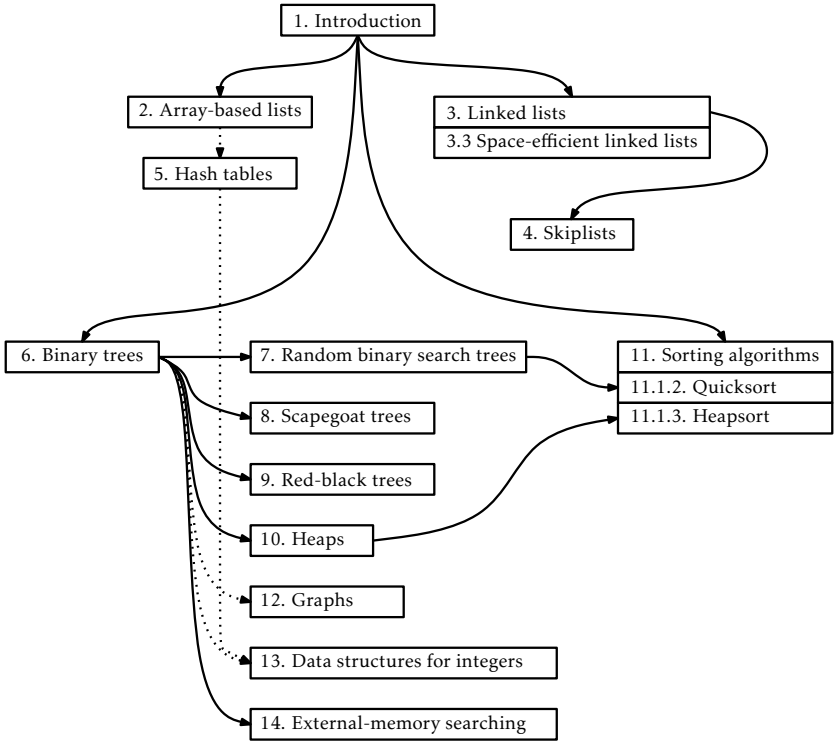

Figure 1.6: The dependencies between chapters in this book.

SSet implementations			
	find(x)	add(x)/remove(x)	
SkiplistSSet	$O(\log n)^{\mathrm{E}}$	$O(\log n)^{\mathrm{E}}$	§ 4.2
Treap	$O(\log n)^{\mathrm{E}}$	$O(\log n)^{\mathrm{E}}$	§ 7.2
ScapegoatTree	$O(\log n)$	$O(\log n)^{\mathrm{A}}$	§ 8.1
RedBlackTree	$O(\log n)$	$O(\log n)$	§ 9.2
BinaryTrie$^{\mathrm{I}}$	$O(w)$	$O(w)$	§ 13.1
XFastTrie$^{\mathrm{I}}$	$O(\log w)^{\mathrm{A,E}}$	$O(w)^{\mathrm{A,E}}$	§ 13.2
YFastTrie$^{\mathrm{I}}$	$O(\log w)^{\mathrm{A,E}}$	$O(\log w)^{\mathrm{A,E}}$	§ 13.3

(Priority) Queue implementations			
	find_min()	add(x)/remove()	
BinaryHeap	$O(1)$	$O(\log n)^{\mathrm{A}}$	§ 10.1
MeldableHeap	$O(1)$	$O(\log n)^{\mathrm{E}}$	§ 10.2

$^{\mathrm{I}}$ This structure can only store w-bit integer data.

Table 1.2: Summary of SSet and priority Queue implementations.

formal definitions of exponentials and logarithms, see the (freely available) classic text by Thompson [71].

For more information on basic probability, especially as it relates to computer science, see the textbook by Ross [63]. Another good reference, which covers both asymptotic notation and probability, is the textbook by Graham, Knuth, and Patashnik [37].

Exercise 1.1. This exercise is designed to help familiarize the reader with choosing the right data structure for the right problem. If implemented, the parts of this exercise should be done by making use of an implementation of the relevant interface (Stack, Queue, Deque, USet, or SSet) provided by the .

Solve the following problems by reading a text file one line at a time and performing operations on each line in the appropriate data structure(s). Your implementations should be fast enough that even files containing a million lines can be processed in a few seconds.

1. Read the input one line at a time and then write the lines out in reverse order, so that the last input line is printed first, then the

second last input line, and so on.

2. Read the first 50 lines of input and then write them out in reverse order. Read the next 50 lines and then write them out in reverse order. Do this until there are no more lines left to read, at which point any remaining lines should be output in reverse order.

   In other words, your output will start with the 50th line, then the 49th, then the 48th, and so on down to the first line. This will be followed by the 100th line, followed by the 99th, and so on down to the 51st line. And so on.

   Your code should never have to store more than 50 lines at any given time.

3. Read the input one line at a time. At any point after reading the first 42 lines, if some line is blank (i.e., a string of length 0), then output the line that occured 42 lines prior to that one. For example, if Line 242 is blank, then your program should output line 200. This program should be implemented so that it never stores more than 43 lines of the input at any given time.

4. Read the input one line at a time and write each line to the output if it is not a duplicate of some previous input line. Take special care so that a file with a lot of duplicate lines does not use more memory than what is required for the number of unique lines.

5. Read the input one line at a time and write each line to the output only if you have already read this line before. (The end result is that you remove the first occurrence of each line.) Take special care so that a file with a lot of duplicate lines does not use more memory than what is required for the number of unique lines.

6. Read the entire input one line at a time. Then output all lines sorted by length, with the shortest lines first. In the case where two lines have the same length, resolve their order using the usual "sorted order." Duplicate lines should be printed only once.

7. Do the same as the previous question except that duplicate lines should be printed the same number of times that they appear in the input.

8. Read the entire input one line at a time and then output the even numbered lines (starting with the first line, line 0) followed by the odd-numbered lines.

9. Read the entire input one line at a time and randomly permute the lines before outputting them. To be clear: You should not modify the contents of any line. Instead, the same collection of lines should be printed, but in a random order.

Exercise 1.2. A *Dyck word* is a sequence of +1's and -1's with the property that the sum of any prefix of the sequence is never negative. For example, $+1,-1,+1,-1$ is a Dyck word, but $+1,-1,-1,+1$ is not a Dyck word since the prefix $+1-1-1<0$. Describe any relationship between Dyck words and Stack push(x) and pop() operations.

Exercise 1.3. A *matched string* is a sequence of {, }, (,), [, and] characters that are properly matched. For example, "{{()[]}}" is a matched string, but this "{{()]}" is not, since the second { is matched with a]. Show how to use a stack so that, given a string of length n, you can determine if it is a matched string in $O(n)$ time.

Exercise 1.4. Suppose you have a Stack, s, that supports only the push(x) and pop() operations. Show how, using only a FIFO Queue, q, you can reverse the order of all elements in s.

Exercise 1.5. Using a USet, implement a Bag. A Bag is like a USet—it supports the add(x), remove(x) and find(x) methods—but it allows duplicate elements to be stored. The find(x) operation in a Bag returns some element (if any) that is equal to x. In addition, a Bag supports the find_all(x) operation that returns a list of all elements in the Bag that are equal to x.

Exercise 1.6. From scratch, write and test implementations of the List, USet and SSet interfaces. These do not have to be efficient. They can be used later to test the correctness and performance of more efficient implementations. (The easiest way to do this is to store the elements in an array.)

Exercise 1.7. Work to improve the performance of your implementations from the previous question using any tricks you can think of. Experiment

and think about how you could improve the performance of add(i, x) and remove(i) in your List implementation. Think about how you could improve the performance of the find(x) operation in your USet and SSet implementations. This exercise is designed to give you a feel for how difficult it can be to obtain efficient implementations of these interfaces.

Chapter 2

Array-Based Lists

In this chapter, we will study implementations of the List and Queue interfaces where the underlying data is stored in an array, called the *backing array*. The following table summarizes the running times of operations for the data structures presented in this chapter:

	get(i)/set(i,x)	add(i,x)/remove(i)
ArrayStack	$O(1)$	$O(n-i)$
ArrayDeque	$O(1)$	$O(\min\{i, n-i\})$
DualArrayDeque	$O(1)$	$O(\min\{i, n-i\})$
RootishArrayStack	$O(1)$	$O(n-i)$

Data structures that work by storing data in a single array have many advantages and limitations in common:

- Arrays offer constant time access to any value in the array. This is what allows get(i) and set(i,x) to run in constant time.
- Arrays are not very dynamic. Adding or removing an element near the middle of a list means that a large number of elements in the array need to be shifted to make room for the newly added element or to fill in the gap created by the deleted element. This is why the operations add(i,x) and remove(i) have running times that depend on n and i.
- Arrays cannot expand or shrink. When the number of elements in the data structure exceeds the size of the backing array, a new array

needs to be allocated and the data from the old array needs to be copied into the new array. This is an expensive operation.

The third point is important. The running times cited in the table above do not include the cost associated with growing and shrinking the backing array. We will see that, if carefully managed, the cost of growing and shrinking the backing array does not add much to the cost of an *average* operation. More precisely, if we start with an empty data structure, and perform any sequence of m add(i,x) or remove(i) operations, then the total cost of growing and shrinking the backing array, over the entire sequence of m operations is $O(m)$. Although some individual operations are more expensive, the amortized cost, when amortized over all m operations, is only $O(1)$ per operation.

2.1 ArrayStack: Fast Stack Operations Using an Array

An ArrayStack implements the list interface using an array a, called the *backing array*. The list element with index i is stored in $a[i]$. At most times, a is larger than strictly necessary, so an integer n is used to keep track of the number of elements actually stored in a. In this way, the list elements are stored in $a[0],\ldots,a[n-1]$ and, at all times, length$(a) \geq n$.

```
initialize()
    a ← new_array(1)
    n ← 0
```

2.1.1 The Basics

Accessing and modifying the elements of an ArrayStack using get(i) and set(i,x) is trivial. After performing any necessary bounds-checking we simply return or set, respectively, $a[i]$.

```
get(i)
    return a[i]
```

```
set(i, x)
    y ← a[i]
    a[i] ← x
    return y
```

The operations of adding and removing elements from an ArrayStack are illustrated in Figure 2.1. To implement the add(i,x) operation, we first check if a is already full. If so, we call the method resize() to increase the size of a. How resize() is implemented will be discussed later. For now, it is sufficient to know that, after a call to resize(), we can be sure that length$(a) > n$. With this out of the way, we now shift the elements $a[i],\ldots,a[n-1]$ right by one position to make room for x, set $a[i]$ equal to x, and increment n.

```
add(i, x)
    if n = length(a) then resize()
    a[i+1, i+2, ..., n] ← a[i, i+1, ..., n-1]
    a[i] ← x
    n ← n + 1
```

If we ignore the cost of the potential call to resize(), then the cost of the add(i,x) operation is proportional to the number of elements we have to shift to make room for x. Therefore the cost of this operation (ignoring the cost of resizing a) is $O(n-i)$.

Implementing the remove(i) operation is similar. We shift the elements $a[i+1],\ldots,a[n-1]$ left by one position (overwriting $a[i]$) and decrease the value of n. After doing this, we check if n is getting much smaller than length(a) by checking if length$(a) \geq 3n$. If so, then we call resize() to reduce the size of a.

```
remove(i)
    x ← a[i]
    a[i, i+1, ..., n-2] ← a[i+1, i+2, ..., n-1]
    n ← n - 1
```

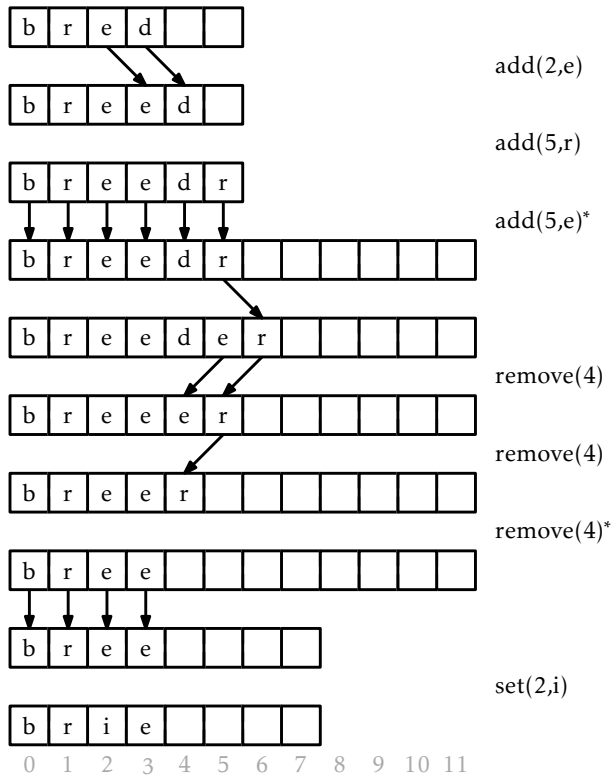

Figure 2.1: A sequence of add(i,x) and remove(i) operations on an ArrayStack. Arrows denote elements being copied. Operations that result in a call to resize() are marked with an asterisk.

```
    if length(a) ≥ 3 · n then resize()
    return x
```

If we ignore the cost of the resize() method, the cost of a remove(i) operation is proportional to the number of elements we shift, which is $O(n-i)$.

2.1.2 Growing and Shrinking

The resize() method is fairly straightforward; it allocates a new array b whose size is $2n$ and copies the n elements of a into the first n positions in b, and then sets a to b. Thus, after a call to resize(), length(a) = $2n$.

```
resize()
    b ← new_array(max(1, 2 · n))
    b[0, 1, ..., n − 1] ← a[0, 1, ..., n − 1]
    a ← b
```

Analyzing the actual cost of the resize() operation is easy. It allocates an array b of size $2n$ and copies the n elements of a into b. This takes $O(n)$ time.

The running time analysis from the previous section ignored the cost of calls to resize(). In this section we analyze this cost using a technique known as *amortized analysis*. This technique does not try to determine the cost of resizing during each individual add(i,x) and remove(i) operation. Instead, it considers the cost of all calls to resize() during a sequence of m calls to add(i,x) or remove(i). In particular, we will show:

Lemma 2.1. *If an empty ArrayStack is created and any sequence of $m \geq 1$ calls to* add(i,x) *and* remove(i) *are performed, then the total time spent during all calls to* resize() *is $O(m)$.*

Proof. We will show that any time resize() is called, the number of calls to *add* or *remove* since the last call to resize() is at least $n/2-1$. Therefore, if n_i denotes the value of n during the ith call to resize() and r denotes

the number of calls to resize(), then the total number of calls to add(i,x) or remove(i) is at least

$$\sum_{i=1}^{r}(n_i/2-1) \le m \ ,$$

which is equivalent to

$$\sum_{i=1}^{r} n_i \le 2m + 2r \ .$$

On the other hand, the total time spent during all calls to resize() is

$$\sum_{i=1}^{r} O(n_i) \le O(m+r) = O(m) \ ,$$

since r is not more than m. All that remains is to show that the number of calls to add(i,x) or remove(i) between the $(i-1)$th and the ith call to resize() is at least $n_i/2$.

There are two cases to consider. In the first case, resize() is being called by add(i,x) because the backing array a is full, i.e., length(a) = $n = n_i$. Consider the previous call to resize(): after this previous call, the size of a was length(a), but the number of elements stored in a was at most length(a)/2 = $n_i/2$. But now the number of elements stored in a is n_i = length(a), so there must have been at least $n_i/2$ calls to add(i,x) since the previous call to resize().

The second case occurs when resize() is being called by remove(i) because length(a) $\ge 3n = 3n_i$. Again, after the previous call to resize() the number of elements stored in a was at least length(a)/2−1.[1] Now there are $n_i \le$ length(a)/3 elements stored in a. Therefore, the number of remove(i) operations since the last call to resize() is at least

$$\begin{aligned} R &\ge \text{length}(a)/2 - 1 - \text{length}(a)/3 \\ &= \text{length}(a)/6 - 1 \\ &= (\text{length}(a)/3)/2 - 1 \\ &\ge n_i/2 - 1 \ . \end{aligned}$$

[1] The -1 in this formula accounts for the special case that occurs when $n = 0$ and length(a) = 1.

In either case, the number of calls to add(i,x) or remove(i) that occur between the $(i-1)$th call to resize() and the ith call to resize() is at least $n_i/2-1$, as required to complete the proof. □

2.1.3 Summary

The following theorem summarizes the performance of an ArrayStack:

Theorem 2.1. *An ArrayStack implements the List interface. Ignoring the cost of calls to* resize(), *an ArrayStack supports the operations*

- get(i) *and* set(i,x) *in* $O(1)$ *time per operation; and*
- add(i,x) *and* remove(i) *in* $O(1+n-i)$ *time per operation.*

Furthermore, beginning with an empty ArrayStack and performing any sequence of m add(i,x) *and* remove(i) *operations results in a total of* $O(m)$ *time spent during all calls to* resize().

The ArrayStack is an efficient way to implement a Stack. In particular, we can implement push(x) as add(n,x) and pop() as remove($n-1$), in which case these operations will run in $O(1)$ amortized time.

2.2 FastArrayStack: An Optimized ArrayStack

Much of the work done by an ArrayStack involves shifting (by add(i,x) and remove(i)) and copying (by resize()) of data. In a naive implementation, this would be done using **for** loops. It turns out that many programming environments have specific functions that are very efficient at copying and moving blocks of data. In the C programming language, there are the memcpy(d,s,n) and memmove(d,s,n) functions. In the C++ language there is the stdcopy(a_0,a_1,b) algorithm. In Java there is the System.arraycopy(s,i,d,j,n) method.

These functions are usually highly optimized and may even use special machine instructions that can do this copying much faster than we could by using a **for** loop. Although using these functions does not asymptotically decrease the running times, it can still be a worthwhile optimization.

In our C++ and Java implementations, the use of fast array copying functions resulted in speedups of a factor between 2 and 3, depending on the types of operations performed. Your mileage may vary.

2.3 ArrayQueue: An Array-Based Queue

In this section, we present the ArrayQueue data structure, which implements a FIFO (first-in-first-out) queue; elements are removed (using the remove() operation) from the queue in the same order they are added (using the add(x) operation).

Notice that an ArrayStack is a poor choice for an implementation of a FIFO queue. It is not a good choice because we must choose one end of the list upon which to add elements and then remove elements from the other end. One of the two operations must work on the head of the list, which involves calling add(i,x) or remove(i) with a value of $i = 0$. This gives a running time proportional to n.

To obtain an efficient array-based implementation of a queue, we first notice that the problem would be easy if we had an infinite array a. We could maintain one index j that keeps track of the next element to remove and an integer n that counts the number of elements in the queue. The queue elements would always be stored in

$$a[j], a[j+1], \ldots, a[j+n-1] \ .$$

Initially, both j and n would be set to 0. To add an element, we would place it in $a[j+n]$ and increment n. To remove an element, we would remove it from $a[j]$, increment j, and decrement n.

Of course, the problem with this solution is that it requires an infinite array. An ArrayQueue simulates this by using a finite array a and *modular arithmetic*. This is the kind of arithmetic used when we are talking about the time of day. For example 10:00 plus five hours gives 3:00. Formally, we say that

$$10 + 5 = 15 \equiv 3 \pmod{12} \ .$$

We read the latter part of this equation as "15 is congruent to 3 modulo 12." We can also treat mod as a binary operator, so that

$$15 \bmod 12 = 3 \ .$$

More generally, for an integer a and positive integer m, a mod m is the unique integer $r \in \{0,\ldots,m-1\}$ such that $a = r + km$ for some integer k. Less formally, the value r is the remainder we get when we divide a by m. In many programming languages, including C, C++, and Java, the mod operate is represented using the % symbol.

Modular arithmetic is useful for simulating an infinite array, since i mod length(a) always gives a value in the range $0,\ldots,$length$(a)-1$. Using modular arithmetic we can store the queue elements at array locations

$$a[j \bmod \text{length}(a)], a[(j+1) \bmod \text{length}(a)], \ldots, a[(j+n-1) \bmod \text{length}(a)] \ .$$

This treats the array a like a *circular array* in which array indices larger than length$(a)-1$ "wrap around" to the beginning of the array.

The only remaining thing to worry about is taking care that the number of elements in the ArrayQueue does not exceed the size of a.

```
initialize()
    a ← new_array(1)
    j ← 0
    n ← 0
```

A sequence of add(x) and remove() operations on an ArrayQueue is illustrated in Figure 2.2. To implement add(x), we first check if a is full and, if necessary, call resize() to increase the size of a. Next, we store x in $a[(j+n) \bmod \text{length}(a)]$ and increment n.

```
add(x)
    if n + 1 > length(a) then resize()
    a[(j + n) mod length(a)] ← x
    n ← n + 1
    return true
```

To implement remove(), we first store $a[j]$ so that we can return it later. Next, we decrement n and increment j (modulo length(a)) by setting $j = (j+1) \bmod \text{length}(a)$. Finally, we return the stored value of $a[j]$. If necessary, we may call resize() to decrease the size of a.

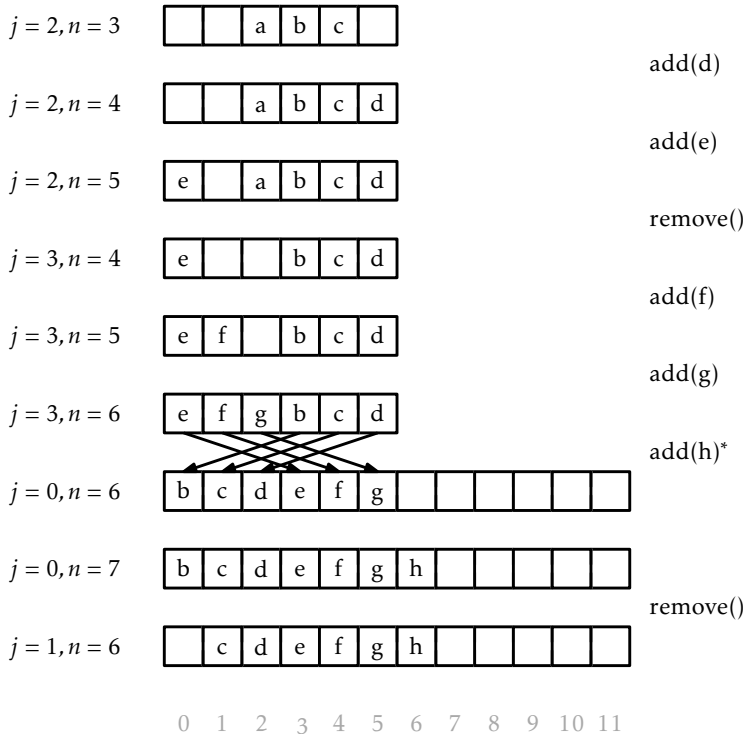

Figure 2.2: A sequence of add(x) and remove(i) operations on an ArrayQueue. Arrows denote elements being copied. Operations that result in a call to resize() are marked with an asterisk.

```
remove()
    x ← a[j]
    j ← (j + 1) mod length(a)
    n ← n − 1
    if length(a) ≥ 3 · n then resize()
    return x
```

Finally, the resize() operation is very similar to the resize() operation of ArrayStack. It allocates a new array, b, of size $2n$ and copies

$$a[j], a[(j+1) \bmod \text{length}(a)], \ldots, a[(j+n-1) \bmod \text{length}(a)]$$

onto

$$b[0], b[1], \ldots, b[n-1]$$

and sets $j = 0$.

```
resize()
    b ← new_array(max(1, 2 · n))
    for k in 0, 1, 2, ..., n − 1 do
        b[k] ← a[(j + k) mod length(a)]
    a ← b
    j ← 0
```

2.3.1 Summary

The following theorem summarizes the performance of the ArrayQueue data structure:

Theorem 2.2. *An ArrayQueue implements the (FIFO) Queue interface. Ignoring the cost of calls to* resize(), *an ArrayQueue supports the operations* add(x) *and* remove() *in $O(1)$ time per operation. Furthermore, beginning with an empty ArrayQueue, any sequence of m* add(i, x) *and* remove(i) *operations results in a total of $O(m)$ time spent during all calls to* resize().

2.4 ArrayDeque: Fast Deque Operations Using an Array

The ArrayQueue from the previous section is a data structure for representing a sequence that allows us to efficiently add to one end of the sequence and remove from the other end. The ArrayDeque data structure allows for efficient addition and removal at both ends. This structure implements the List interface by using the same circular array technique used to represent an ArrayQueue.

```
initialize()
    a ← new_array(1)
    j ← 0
    n ← 0
```

The get(i) and set(i,x) operations on an ArrayDeque are straightforward. They get or set the array element $a[(j+i) \bmod \text{length}(a)]$.

```
get(i)
    return a[(i + j) mod length(a)]

set(i,x)
    y ← a[(i + j) mod length(a)]
    a[(i + j) mod length(a)] ← x
    return y
```

The implementation of add(i,x) is a little more interesting. As usual, we first check if a is full and, if necessary, call resize() to resize a. Remember that we want this operation to be fast when i is small (close to 0) or when i is large (close to n). Therefore, we check if $i < n/2$. If so, we shift the elements $a[0],\ldots,a[i-1]$ left by one position. Otherwise ($i \geq n/2$), we shift the elements $a[i],\ldots,a[n-1]$ right by one position. See Figure 2.3 for an illustration of add(i,x) and remove(x) operations on an ArrayDeque.

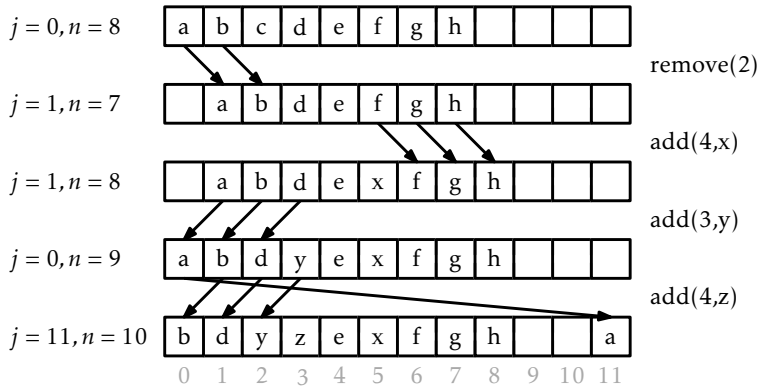

Figure 2.3: A sequence of add(i,x) and remove(i) operations on an ArrayDeque. Arrows denote elements being copied.

```
add(i,x)
    if n = length(a) then resize()
    if i < n/2 then
        j ← (j − 1) mod length(a)
        for k in 0,1,2,...,i − 1 do
            a[(j + k) mod length(a)] ← a[(j + k + 1) mod length(a)]
    else
        for k in n,n − 1,n − 2,...,i + 1 do
            a[(j + k) mod length(a)] ← a[(j + k − 1) mod length(a)]
    a[(j + i) mod length(a)] ← x
    n ← n + 1
```

By doing the shifting in this way, we guarantee that add(i,x) never has to shift more than $\min\{i, n-i\}$ elements. Thus, the running time of the add(i,x) operation (ignoring the cost of a resize() operation) is $O(1 + \min\{i, n-i\})$.

The implementation of the remove(i) operation is similar. It either shifts elements $a[0],\ldots,a[i-1]$ right by one position or shifts the elements $a[i+1],\ldots,a[n-1]$ left by one position depending on whether $i < n/2$. Again, this means that remove(i) never spends more than $O(1 + \min\{i, n-$